Designing and Querying a Clinical Trial Participant Registry

Westbrook University Hospitals NHS Trust — Clinical Trial Participant Registry

Malik Lainsbury

2026-06-15

Table of contents

Introduction

This report designs a relational database for the Westbrook University Hospitals NHS Trust Clinical Trial Participant Registry, implements it in Microsoft SQL Server (T-SQL), populates it, and answers seven clinical/research questions. The accompanying runnable script is `script/HDS_DB_LAINSBURY_2026.sql`.

> **Note**
>
> **Reproducing the screenshots.** Execute the scripts in order (`00 → 01 → 02 → 03 → 05 → 04`) or run the single combined file `HDS_DB_LAINSBURY_2026.sql` against the SQL Server connection in VS Code. Each query returns one result grid; capture one screenshot per grid and place it in the matching slot. The *expected result* listed under each query lets you confirm correctness.

1 Part 1 — Database Design

1.1 Entity Relationship Diagram

The schema has eight tables. `TrialSite` resolves the many-to-many relationship between trials and sites; `Enrollment` records a participant's participation in a trial at a site; `ConsentSigning` is the append-only log of consent events (Figure 1).

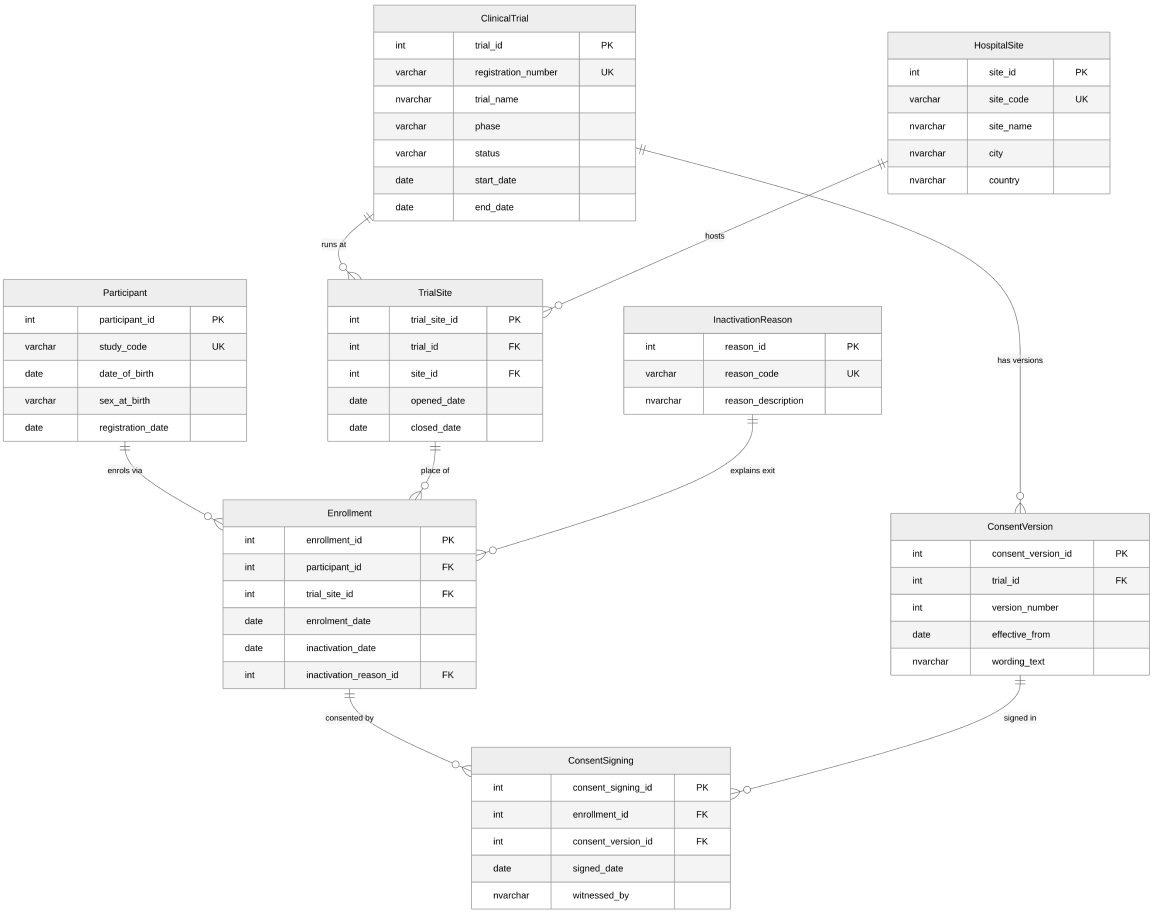

Figure 1: Entity Relationship Diagram (crow's-foot notation).

1.2 Table Descriptions

Every table uses a surrogate `INT IDENTITY` primary key for stable internal joins, plus one or more **business (natural) UNIQUE keys** for the values clinical staff actually recognise.

Table 1: Table descriptions and key design choices

Table	Purpose and key design choices
ClinicalTrial	One row per trial. PK `trial_id` (IDENTITY). UNIQUE `registration_number` — the ISRCTN number is the externally recognised, never-reused identifier, so it is the natural key. `CHECK` on `phase` (I/II/III/IV) and `status` enforce the controlled vocabularies; `CHECK (end_date >= start_date)` prevents impossible ranges.
HospitalSite	One row per site. PK `site_id`. UNIQUE `site_code` — short codes (e.g. `WBK-GEN`) are how sites are referred to operationally, so they form the natural key.
Participant	One pseudonymised participant per row; **no names held**. PK `participant_id`. UNIQUE `study_code` — the pseudonym is the only safe external identifier. `CHECK` on `sex_at_birth`; `CHECK (date_of_birth < registration_date)`.
InactivationReason	Controlled vocabulary of reasons a participant leaves a trial, so exits report consistently across trials. PK `reason_id`. UNIQUE `reason_code` — the stable report token.
TrialSite	**Link table** resolving the trial–site many-to-many; also records when each site opened/closed for recruitment on that trial. PK `trial_site_id`. `UNIQUE (trial_id, site_id)` is the real business key; FKs to ClinicalTrial and HospitalSite; `CHECK (closed_date >= opened_date)`.
ConsentVersion	Every approved version of a trial's consent form, including wording and effective date; old versions never overwritten. PK `consent_version_id`. `UNIQUE (trial_id, version_number)`; FK to ClinicalTrial; `wording_text` is `NVARCHAR(MAX)`.
Enrollment	A participant enrolled in a trial at a specific site. Points at a `TrialSite` row so trial **and** site are captured together and a participant can only enrol where the trial runs. Exit is recorded without deleting the row. PK `enrollment_id`; FKs to Participant, TrialSite, InactivationReason; `UNIQUE (participant_id, trial_site_id)`; paired-null `CHECK`; filtered unique index for "one active trial at a time".
ConsentSigning	Append-only log of every consent signing event: when, which version, who witnessed. PK `consent_signing_id`; FKs to Enrollment and ConsentVersion; `UNIQUE (enrollment_id, consent_version_id)`; protected by an append-only trigger.

Check constraints. Phase/status/sex use `CHECK ... IN (...)` to guarantee clean categorical data for cross-trial reporting. Date checks (`end_date >= start_date`, `closed_date >= opened_date`, `inactivation_date >= enrolment_date`) prevent logically impossible records. The paired-null check on `Enrollment` guarantees a leaving date never appears without a reason and vice-versa.

1.3 NULL and NOT NULL Decisions

Table 2: Nullability decisions and their justification

Column	Decision	Meaning of NULL / rationale
`ClinicalTrial.end_date`	NULL	NULL is meaningful: the trial is still running. Current trials have no end date *yet*.
`ClinicalTrial.start_date`, `status`, `phase`	NOT NULL	A trial cannot exist without these; absence would be an error, not a real state.
`TrialSite.closed_date`	NULL	NULL = the site is still open for recruitment on that trial.
`Enrollment.inactivation_date` & `inactivation_reason_id`	NULL (paired)	The classic "meaningful absence": NULL = still active. Both fill in on exit; the paired-null `CHECK` keeps them in step.
`Participant.date_of_birth`, `registration_date`, `sex_at_birth`	NOT NULL	Recorded at registration and required for eligibility, age and demographic reporting.
`ConsentSigning.signed_date`, `witnessed_by`	NOT NULL	A consent event without a date or witness is not a valid consent event.
`ConsentVersion.wording_text`, `effective_from`	NOT NULL	A consent version is meaningless without its wording and the date it took effect.

1.4 Relationships Between Tables

Table 3: Relationships between tables

Tables	Type / FK	What it represents
ClinicalTrial – HospitalSite	M:N via `TrialSite` (FK `trial_id`, `site_id`)	A trial runs at many sites and a site hosts many trials; the link table also carries the open/close recruitment dates.
ClinicalTrial -> ConsentVersion	1:M (FK `trial_id`)	One trial accumulates many consent-form versions over its life as the protocol changes.
Participant -> Enrollment	1:M (FK `participant_id`)	A participant may have several enrolments over time, only one active at once (filtered index).
TrialSite -> Enrollment	1:M (FK `trial_site_id`)	Each enrolment happens at exactly one trial-site.
InactivationReason -> Enrollment	1:M optional (FK `inactivation_reason_id`)	A standardised reason classifies many exits; active enrolments reference none.
Enrollment -> ConsentSigning	1:M (FK `enrollment_id`)	One enrolment can have several signing events (original + re-consent after amendment).

Tables	Type / FK	What it represents
ConsentVersion -> ConsentSigning	1:M (FK `consent_version_id`)	Each signing records which version was signed; one version is signed by many participants.

Many-to-many resolution. The only many-to-many relationship (ClinicalTrial – HospitalSite) is resolved with the `TrialSite` junction table, which carries its own surrogate key, a `UNIQUE (trial_id, site_id)` business key, and the recruitment open/close dates that are attributes of the *pairing* rather than of either entity alone.

2 Part 2 — SQL

2.1 INSERT statements — one per table

All foreign-key values are resolved with subqueries on business keys — no hard-coded IDENTITY integers. The remaining INSERTs are in `02_insert_reference_data.sql` and `03_insert_operational_data.sql`.

ClinicalTrial

```
INSERT INTO dbo.ClinicalTrial (registration_number, trial_name, phase, status,
↪  start_date, end_date)
VALUES ('ISRCTN10234567', N'CARDIOPROTECT: Cardiac Rehabilitation in Post-MI
↪  Patients',
        'III', 'active', '2023-03-01', NULL);
```

HospitalSite

```
INSERT INTO dbo.HospitalSite (site_code, site_name, city, country)
VALUES ('WBK-GEN', N'Westbrook General Hospital', N'London', N'England');
```

Participant

```
INSERT INTO dbo.Participant (study_code, date_of_birth, sex_at_birth,
↪  registration_date)
VALUES ('WBK-001', '1958-04-12', 'Male', '2021-11-03');
```

InactivationReason

```
INSERT INTO dbo.InactivationReason (reason_code, reason_description)
VALUES ('consent_withdrawn', N'Participant withdrew their consent to take part
↪  in the trial');
```

TrialSite (FK look-ups via subquery)

```
INSERT INTO dbo.TrialSite (trial_id, site_id, opened_date, closed_date)
VALUES (
 (SELECT trial_id FROM dbo.ClinicalTrial WHERE
↪  registration_number='ISRCTN10234567'),
 (SELECT site_id  FROM dbo.HospitalSite  WHERE site_code='WBK-GEN'),
 '2023-03-01', NULL);
```

ConsentVersion

```
INSERT INTO dbo.ConsentVersion (trial_id, version_number, effective_from,
↪  wording_text)
VALUES (
 (SELECT trial_id FROM dbo.ClinicalTrial WHERE
↪  registration_number='ISRCTN10234567'),
 1, '2023-03-01',
 N'I agree to take part in the CARDIOPROTECT study. ... I may withdraw at any
↪  time without affecting my medical care.');
```

Enrollment (resolves the trial-site from registration number + site code)

```
INSERT INTO dbo.Enrollment (participant_id, trial_site_id, enrolment_date,
↪  inactivation_date, inactivation_reason_id)
VALUES (
 (SELECT participant_id FROM dbo.Participant WHERE study_code='WBK-001'),
 (SELECT ts.trial_site_id FROM dbo.TrialSite ts
    JOIN dbo.ClinicalTrial ct ON ct.trial_id = ts.trial_id
    JOIN dbo.HospitalSite  hs ON hs.site_id  = ts.site_id
    WHERE ct.registration_number='ISRCTN10234567' AND hs.site_code='WBK-GEN'),
 '2023-03-15', NULL, NULL);
```

ConsentSigning (resolves enrolment + version by business keys)

```
INSERT INTO dbo.ConsentSigning (enrollment_id, consent_version_id, signed_date,
↪  witnessed_by)
VALUES (
 (SELECT e.enrollment_id FROM dbo.Enrollment e
    JOIN dbo.Participant p ON p.participant_id = e.participant_id
    JOIN dbo.TrialSite  ts ON ts.trial_site_id = e.trial_site_id
    JOIN dbo.ClinicalTrial ct ON ct.trial_id = ts.trial_id
    WHERE p.study_code='WBK-001' AND ct.registration_number='ISRCTN10234567'),
 (SELECT cv.consent_version_id FROM dbo.ConsentVersion cv
    JOIN dbo.ClinicalTrial ct ON ct.trial_id = cv.trial_id
    WHERE ct.registration_number='ISRCTN10234567' AND cv.version_number=1),
 '2023-03-15', N'Nurse A. Okafor');
```

2.2 SQL Queries

2.2.1 Query 1 — Current consent wording for CARDIOPROTECT

Clinical question. Before a consent discussion, a research nurse needs the current (most recently effective) consent wording for CARDIOPROTECT: version number, effective date, and full wording.

```
SELECT TOP (1) cv.version_number, cv.effective_from, cv.wording_text
FROM   dbo.ConsentVersion cv
JOIN   dbo.ClinicalTrial  ct ON ct.trial_id = cv.trial_id
WHERE  ct.trial_name LIKE 'CARDIOPROTECT%'
ORDER  BY cv.effective_from DESC;
```

Screenshot: *[paste output grid here]*

Expected result: version 2, effective `2024-01-15`, the wearable-devices wording.

Discussion. The query filters to CARDIOPROTECT, orders versions by `effective_from` descending and keeps the top row, so it always returns the latest version even after future amendments. *Data quality:* it relies on `effective_from` being accurate — a new version entered with a wrong or missing effective date could show the nurse out-of-date wording.

2.2.2 Query 2 — Monthly trial status report

Clinical question. A summary of each trial, its status, how long it has run in days, and whether it has an end date or is ongoing; active trials first, then recruiting, then the rest.

```
SELECT ct.trial_name, ct.status,
       DATEDIFF(DAY, ct.start_date, COALESCE(ct.end_date, CAST(GETDATE() AS
↪  DATE))) AS days_running,
       CASE WHEN ct.end_date IS NULL THEN 'Ongoing' ELSE 'Ended' END AS
       ↪  end_date_status
FROM   dbo.ClinicalTrial ct
ORDER  BY CASE ct.status WHEN 'active' THEN 1 WHEN 'recruiting' THEN 2 ELSE 3
↪  END,
         ct.trial_name;
```

Screenshot: *[paste output grid here]*

Expected result (3 rows): CARDIOPROTECT (active, Ongoing) first; BREATHE (recruiting, Ongoing) second; MOBILISE (completed, Ended, ~1460 days) last.

Discussion. `COALESCE` substitutes today's date for trials with no end date so "days running" is meaningful for ongoing trials, and a `CASE` expression in `ORDER BY` gives the custom status priority. *Data quality:* "days running" for ongoing trials changes every day the report is run, so it is only a dated snapshot.

2.2.3 Query 3 — Trial history for participant WBK-003

Clinical question. Full history for WBK-003: every trial, enrolment date, inactivation date, and reason code; ordered by enrolment date. Active enrolments have no reason.

```
SELECT ct.trial_name, e.enrolment_date, e.inactivation_date, ir.reason_code
FROM   dbo.Enrollment       e
JOIN   dbo.Participant      p  ON p.participant_id = e.participant_id
JOIN   dbo.TrialSite        ts ON ts.trial_site_id = e.trial_site_id
JOIN   dbo.ClinicalTrial    ct ON ct.trial_id      = ts.trial_id
LEFT   JOIN dbo.InactivationReason ir ON ir.reason_id =
↪  e.inactivation_reason_id
WHERE  p.study_code = 'WBK-003'
ORDER  BY e.enrolment_date;
```

Screenshot: *[paste output grid here]*

Expected result (2 rows): BREATHE, 2024-06-10, 2024-09-15, `consent_withdrawn`; then CARDIOPROTECT, 2024-10-01, NULL, NULL.

Discussion. A `LEFT JOIN` to `InactivationReason` is essential: an active enrolment has a NULL reason and an INNER join would silently drop it. *Data quality:* a participant who left but whose `inactivation_reason_id` was never populated would show a leaving date with a blank reason — the paired-null check constraint prevents exactly this.

2.2.4 Query 4 — Audit of currently active enrolments

Clinical question. Each currently **active** participant with their trial, site, and enrolment date; ordered by trial name then enrolment date.

```
SELECT p.study_code, ct.trial_name, hs.site_name, e.enrolment_date
FROM   dbo.Enrollment    e
JOIN   dbo.Participant   p  ON p.participant_id = e.participant_id
JOIN   dbo.TrialSite     ts ON ts.trial_site_id = e.trial_site_id
JOIN   dbo.ClinicalTrial ct ON ct.trial_id      = ts.trial_id
JOIN   dbo.HospitalSite  hs ON hs.site_id        = ts.site_id
WHERE  e.inactivation_date IS NULL
ORDER  BY ct.trial_name, e.enrolment_date;
```

Screenshot: *[paste output grid here]*

Expected result (4 rows): WBK-005 / BREATHE / Westbrook General / 2024-08-05; WBK-001 / CARDIOPROTECT / Westbrook General / 2023-03-15; WBK-003 / CARDIOPROTECT / Westbrook General / 2024-10-01; WBK-002 / CARDIOPROTECT / St Katherine's / 2025-01-10.

Discussion. "Active" is expressed as `inactivation_date IS NULL`; joining through `TrialSite` to both the trial and the site reconstructs where each active participant sits. *Data quality:* the definition of "active" depends on inactivation being recorded promptly — a participant who has left but whose exit was not entered would wrongly appear here.

2.2.5 Query 5 — Enrolment count per site (including empty sites)

Clinical question. How many participants have ever been enrolled at each site, including sites with none. Show site name, city, total count, descending.

```
SELECT hs.site_name, hs.city, COUNT(e.enrollment_id) AS total_enrolments
```

```
FROM   dbo.HospitalSite hs
LEFT   JOIN dbo.TrialSite  ts ON ts.site_id      = hs.site_id
LEFT   JOIN dbo.Enrollment e  ON e.trial_site_id = ts.trial_site_id
GROUP  BY hs.site_name, hs.city
ORDER  BY total_enrolments DESC;
```

Screenshot: *[paste output grid here]*

Expected result (3 rows): Westbrook General (London) 4; St Katherine's (Manchester) 3; Ridgeway Community (Bristol) 0.

Discussion. Starting from `HospitalSite` with `LEFT JOIN`s preserves sites with no enrolments, and `COUNT(e.enrollment_id)` counts rows rather than NULLs so empty sites correctly score 0. *Data quality:* a site that participates in a trial but has not yet recorded any enrolments is indistinguishable from one that genuinely has none — the zero needs interpretation in context.

2.2.6 Query 6 — Trials with an amended protocol

Clinical question. Trials where the protocol was amended (more than one consent version): trial name, current status, number of versions; descending; only where more than one version exists.

```
SELECT ct.trial_name, ct.status, COUNT(cv.consent_version_id) AS version_count
FROM   dbo.ClinicalTrial  ct
JOIN   dbo.ConsentVersion cv ON cv.trial_id = ct.trial_id
GROUP  BY ct.trial_name, ct.status
HAVING COUNT(cv.consent_version_id) > 1
ORDER  BY version_count DESC;
```

Screenshot: *[paste output grid here]*

Expected result (1 row): CARDIOPROTECT, active, 2.

Discussion. Grouping by trial and counting versions, then filtering with `HAVING COUNT(...) > 1`, isolates amended trials (`HAVING` filters the aggregate, which `WHERE` cannot). *Data quality:* this counts version *rows*, so a duplicate version entered in error would overstate the number of genuine amendments.

2.2.7 Query 7 — Most recently enrolled participant per site

Clinical question. The most recently enrolled participant at each site across all trials: study code, trial name, site name, enrolment date.

```
WITH RankedEnrolments AS (
    SELECT p.study_code, ct.trial_name, hs.site_name, e.enrolment_date,
           ROW_NUMBER() OVER (PARTITION BY hs.site_id
                              ORDER BY e.enrolment_date DESC, e.enrollment_id
                              ↪  DESC) AS rn
    FROM   dbo.Enrollment    e
    JOIN   dbo.Participant   p  ON p.participant_id = e.participant_id
    JOIN   dbo.TrialSite     ts ON ts.trial_site_id = e.trial_site_id
```

```
    JOIN   dbo.ClinicalTrial ct ON ct.trial_id      = ts.trial_id
    JOIN   dbo.HospitalSite  hs ON hs.site_id        = ts.site_id
)
SELECT study_code, trial_name, site_name, enrolment_date
FROM   RankedEnrolments
WHERE  rn = 1
ORDER  BY site_name;
```

Screenshot: *[paste output grid here]*

Expected result (2 rows): WBK-002 / CARDIOPROTECT / St Katherine's Hospital / 2025-01-10; WBK-003 / CARDIOPROTECT / Westbrook General Hospital / 2024-10-01. (Ridgeway has no enrolments, so it does not appear.)

Discussion. `ROW_NUMBER()` partitioned by site and ordered by enrolment date descending ranks each site's enrolments; keeping `rn = 1` returns the newest per site, with `enrollment_id` as a deterministic tie-breaker. *Data quality:* if two participants share the same enrolment date the tie-break is arbitrary, so "most recent" may need a finer timestamp than a date to be unambiguous.

3 Part 3 — Reflection

3.1 Why no participant names?

Storing only a `study_code` pseudonymises the registry: it minimises the personal data held (a GDPR / Caldicott principle) and means a breach of the trial database does not directly expose identities. Direct identifiers such as names live in a separate, access-controlled master linkage list held by the research office, not in the analytical database. In practice this shapes how staff work. Researchers query and report entirely on `study_code`, so trial analyses can be shared more freely because they carry no obvious identifiers. Clinical staff who need to act on a real patient — to contact them or update their medical record — must re-identify the person through the controlled linkage list, which is deliberately a separate, auditable step. The cost is friction and a dependency on that linkage being accurate and available: a broken or out-of-date mapping could mean a participant cannot be reached for a safety issue. The design therefore trades day-to-day convenience for a much stronger confidentiality and information-governance position.

3.2 A situation where the data could mislead

Consider Query 4 and Query 5, which both depend on `inactivation_date`. If a participant has actually left a trial but their exit has not yet been entered, they will still appear as "active" and will be counted in active-enrolment reports — overstating recruitment and potentially prompting unnecessary follow-up contact. The data is not *wrong* field by field; it is *stale*, which is harder to spot. A data manager could detect this by reconciling the registry against source records: cross-checking active enrolments against clinic attendance or against the standardised `InactivationReason` log, and flagging participants with no recent activity (e.g. no consent or visit events for an unusually long period) for review. Preventively, they could add a periodic data-quality report listing long-inactive "active" enrolments, set a service standard for how quickly exits must be recorded, and use the `lost_to_follow_up` reason code consistently so genuine non-contact is captured rather than left as a silent gap. The fix is process plus monitoring, not just a query.

3.3 Why consent records are append-only, and how to enforce it

Consent records are the legal and ethical evidence that each participant agreed to the specific protocol version in force at the time. They must be append-only because consent is a historical fact: editing or deleting a signing event would destroy the audit trail that regulators, sponsors and ethics committees rely on to prove valid consent existed. If the rule were broken, the database would lose referential trust — a re-consent after a protocol amendment could be silently overwritten, a withdrawn consent erased, and it would become impossible to demonstrate which wording a participant actually agreed to. The integrity of every downstream analysis that assumes valid consent would be undermined.

I would enforce this at the database level with an **`INSTEAD OF UPDATE, DELETE` trigger** on `ConsentSigning` that rejects any such operation with an error, while leaving `INSERT` free (see `05_append_only_trigger.sql`):

```
CREATE OR ALTER TRIGGER dbo.trg_ConsentSigning_AppendOnly
ON dbo.ConsentSigning
```

```
INSTEAD OF UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    THROW 51000, 'ConsentSigning is append-only: rows cannot be updated or
↪   deleted.', 1;
END;
```

This guarantees the rule regardless of which application or user connects, which a permission grant alone would not (an administrator could still bypass column permissions). For even stronger, tamper-evident assurance the table could be defined as a SQL Server **temporal (system-versioned) table** or audited so that any attempted change is itself recorded.

Appendix — File manifest

Table 4: File manifest

File	Contents
`script/00_create_database.sql`	(Re)creates the `HDS_ClinicalTrial` database
`script/01_create_tables.sql`	CREATE TABLEs, constraints, indexes
`script/02_insert_reference_data.sql`	The five reference sheets from the brief
`script/03_insert_operational_data.sql`	TrialSite, Enrollment, ConsentSigning data
`script/04_queries.sql`	The seven queries
`script/05_append_only_trigger.sql`	Append-only enforcement for `ConsentSigning`
`script/HDS_DB_LAINSBURY_2026.sql`	**Submission file** — everything in run order

Use of generative AI: *state your declaration here per the coversheet.*